Methodology

Modeling Framework

The robot statics are modeled using SoRoSim, a MATLAB-based toolbox implementing the Geometric Variable-Strain (GVS) formulation of Cosserat rod theory for continuum robots. This framework was selected for its computational efficiency and native support for tendon-driven actuation.

Model Configuration

is this the length?

why 20?

The robot backbone comprises a 200 mm continuum link discretized into 20 equally-spaced segments. Each segment possesses an independently specified radius, allowing arbitrary cross-sectional profiles along the length. The backbone material properties correspond to PLA (Young's modulus E = 2.9 GPa, Poisson's ratio v = 0.34). The distal end is unconstrained with three degrees of freedom: two-axis bending and axial extension. The proximal end is fixed to the global coordinate frame origin.

Is this from Bamboo data sheet?

a bit confused by this... your robot has a fixed length?

To quantify reaction forces and moments at the base, a six-degree-of-freedom constraint (three translational, three rotational) is enforced at the proximal end. This constraint is position-controlled, fixed at a specified position and orientation by the user. SoRoSim then allows us to solve the six-dimensional wrench required to maintain this specified fixed configuration.

Tendon Actuation Model

Three tendons are routed along straight, constant-radius paths at 120° circumferential intervals with an 8 mm radial offset from the backbone centerline. Each tendon path is modeled as a constraint curve rather than through explicit routing disk geometry. This simplification assumes the physical routing disks are sufficiently thin that their bending stiffness contribution is negligible compared to the backbone, reducing model complexity while maintaining computational efficiency.

Each tendon is independently actuated via tension control $[\tau_1, \tau_2, \tau_3]$. Importantly, tendon actuation in isolation does not generate net reaction wrench at the proximal constraint. rather, it modifies the backbone equilibrium configuration, thereby altering the mapping between applied external loads and measured base reactions.

Applied Point Forces

can you provide a sketch?

Applied forces are modeled as follower point forces (i.e., orientation is relative to the local frame at the application point, not the global frame). Each point force is defined by four components $[F, s, el, az]$:

- F: magnitude
- s: position along the total arc length
- el: elevation angle relative to the local x-y plane
- az: azimuth angle relative to the local y-z plane

These four components fully describe a point force acting on the robot in the local frame.

Since point loads can produce singular stress concentrations that ill-condition the system and impede solver convergence, we approximate point loads using narrow Gaussian distributions, regularizing the problem and enabling stable, reliable convergence.

→ details? what are parameters? How were they chosen?

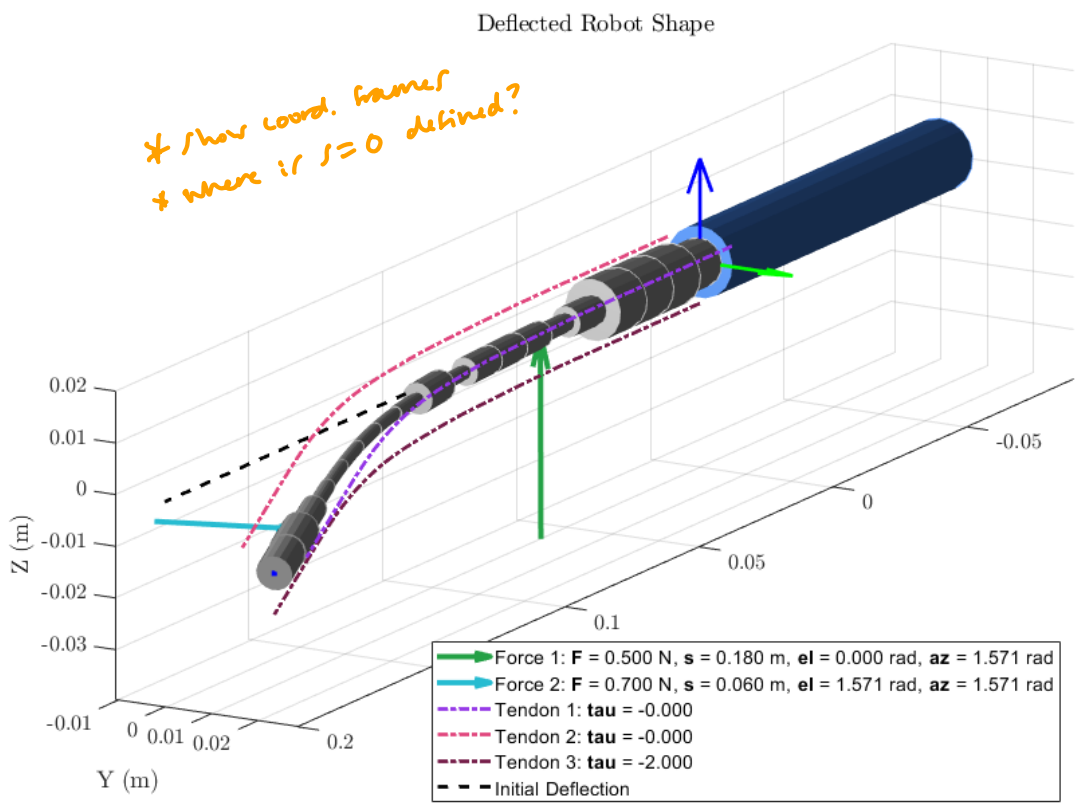
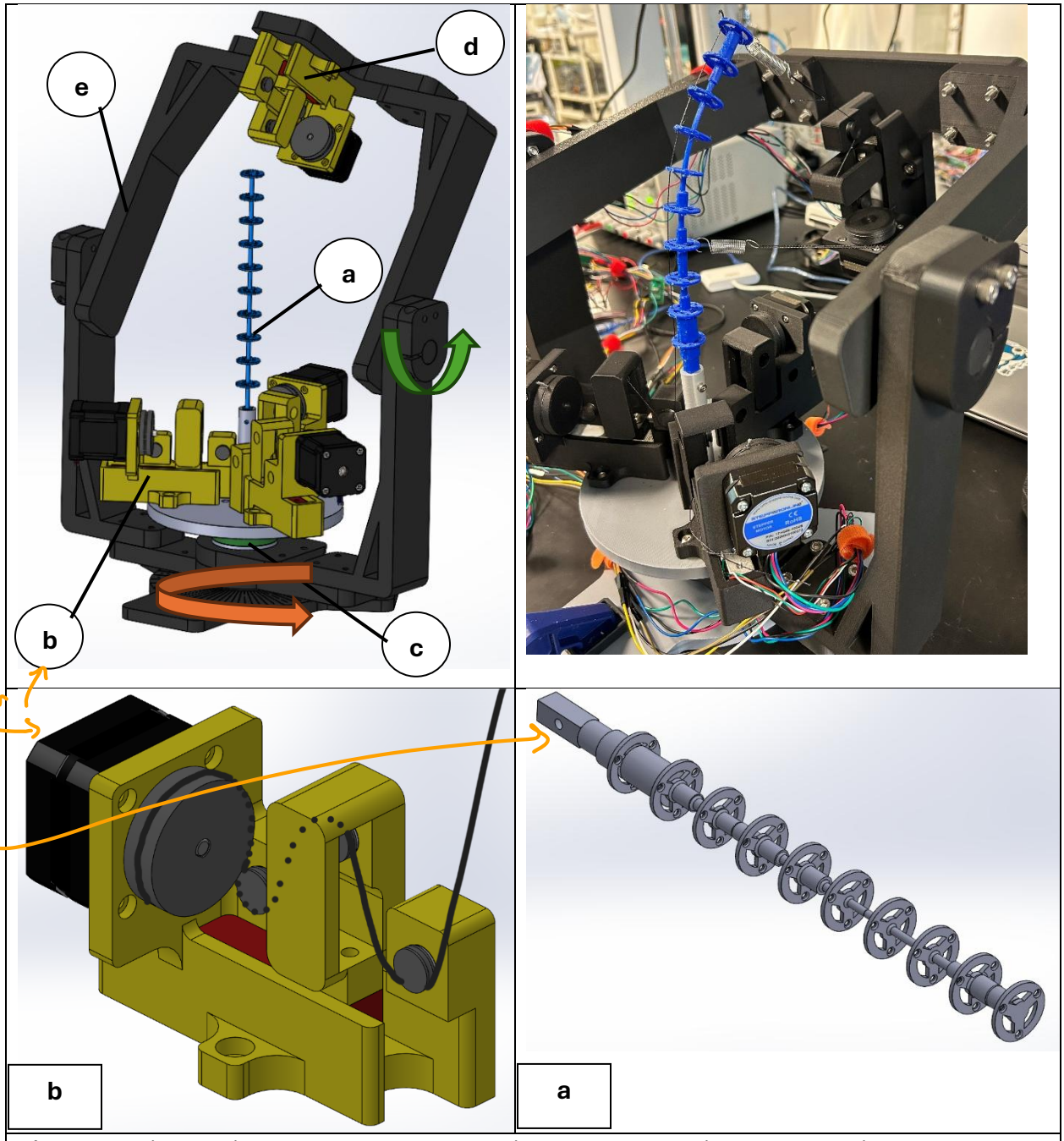


Figure: Robot configuration in SoRoSim showing variable radii, tendon routing, two applied forces, and resulting deflection

Experimental Setup

Robot Setup

To compare the force estimation performance of different geometric designs, we built a motorized testing setup to apply two point forces and apply three tendon tensions.



would be good to show blown up view in same orientation as main fig

Figure: (a) link design, (b) closed-loop tension-controlled winch for applying tendon tensions, (c) 6-axis force-torque sensor, (d) closed-loop tension-controlled winch for applying point force, (e) adjustable frame with two axes of rotation.

Three winches positioned at 120° intervals around the circumference of the designated beam enable independent closed-loop tension control of each tendon. Each winch uses a NEMA 17 stepper motor and TMC2209 stepper motor driver to wind the tendon in and out.

The tendon wraps around a pulley configuration that directs it through a load cell. When tensioned, the tendon pulls a center piece into the load cell, which measures a force component proportional to the total tension (the proportion depends on the pulley angles). The system was calibrated using known masses.

The load cell provides high-bandwidth feedback to an Arduino Mega microcontroller running a proportional control algorithm that reduces motor speed as tension approaches the target value.

The adjustable frame provides three degrees of freedom: two rotations and translational adjustment of the force application point by repositioning the hook attachment along the beam's length. An additional winch mounted on the upper frame applies a target external point force. Motor tension targets are set via the Arduino IDE serial interface.

Design of Experiment

For hardware validation we tested two designs:

Design	**Length (mm)**	**Radius (mm)**
Cylindrical	20	1.75
Optimized	20	[1.25, 6]

For each design, we conducted 40 experimental trials across a range of force inputs. We selected force inputs using JMP's custom design of experiment (DOE), which provides uniform sampling of the input space, identifies the most informative variable combinations, and achieves equivalent accuracy with fewer trials than traditional methods.

The factor levels selected for the experiment are shown below.

Factor name	**# of factor levels**	**Levels**
F_1	10	0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0
s_1	6	0.1, 0.12, 0.14, 0.16, 0.18, 0.2
el_1	5	90°, 135°, 180°, 225°, 270°
F_2	10	0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0
s_2	4	0.2, 0.4, 0.6, 0.8
el_2	3	180°, 225°, 270°
tau_1	8	0, 0.2, 0.4, 0.6, 0.8, 1.0, 1.2, 1.4

Forward Model

is this a built in function?

The forward model is computed using:

$$W(x,\tau) = get_proximal_value(x,\tau) \in \mathbb{R}^6$$

Input: Applied force $x = [F, s, el, az]$ and tendon tensions $\tau = [\tau_1, \tau_2, \tau_3]$

Output: Resulting proximal wrench $W = [T_x, T_y, T_z, F_x, F_y, F_z]$ computed by the SoRoSim statics() function

Computation time: Approximately 0.05 s

Multiple applied forces:

For N applied point forces, stack vertically $x = \begin{bmatrix} F_1, s_1, el_1, az_1 \\ \vdots \\ F_N, s_N, el_N, az_N \end{bmatrix} \mathbb{R}^{Nx4}$

Inverse Model

Single Point Force

Is this a built-in function?

The inverse model for a single point force is computed using:

$$\hat{x} = [\hat{F}, \hat{s}, \widehat{el}, \widehat{az}] = get_distal_value(W_m, \tau_m)$$

Input: Measured proximal wrench $W_m = [T_x, T_y, T_z, F_x, F_y, F_z]$ (via ATI Mini40 IP65 6-axis force-torque sensor) and measured tendon tensions $\tau_m = [\tau_1, \tau_2, \tau_3]$ (via load cells)

Output: Estimated external applied force $\hat{x} = [\hat{F}, \hat{s}, \widehat{el}, \widehat{az}]$

The inverse model employs a shooting method to minimize a residual:

$$\hat{x} = \underset{l \le x \le u}{\operatorname{argmin}} \frac{1}{2} \|r\|_2^2$$

where the residual is:

notation is a bit confusing?

How is this selected?

$$r(x) = S_6(W(x,\tau_m) - W_m)$$

Upper and lower bounds l and u are defined for the four design variables, an initial guess x_0 is selected, and a nonlinear least-squares optimizer using the trust-region-reflective algorithm is applied. The optimizer computes the forward model $W(x,\tau_m)$ and compares it to the measured target W_m to form the residual with weights/normalization $S_6 \in \mathbb{R}^6$. The optimizer iteratively seeks design variables that minimize the residual until an acceptable optimality tolerance is achieved.

How were these tuned?

Two Point Forces

The inverse model for two point forces is computed using:

$$\hat{x} = \left[\widehat{F_1}, \widehat{s_1}, \widehat{el_1};\ \widehat{F_2}, \widehat{s_2}, \widehat{el_2}\right] = get_distal_value(W_m, \tau_m)$$

Input: Same as single point force model

Output: Estimated two external applied forces $\hat{x} = \left[\widehat{F_1}, \widehat{s_1}, \widehat{el_1};\ \widehat{F_2}, \widehat{s_2}, \widehat{el_2}\right]$

This model assumes both point forces are locally perpendicular to the backbone $\left(az = \frac{pi}{2}\right)$.

Iterative Loading Method

To determine the number of point forces without camera information, we use an iterative approach. The method begins by minimizing the residual norm with a single point force, then iteratively adds forces until no further improvement is observed. This final configuration identifies the optimal number of forces for the loading case.

This approach is particularly effective when forces have significantly different magnitudes. When small and large forces are applied simultaneously, the proximal wrench of the larger force can dominate the signal, causing the optimizer to incorrectly model two equal forces at the larger force location. The iterative method avoids this pitfall by recognizing when a single force provides a better fit, effectively filtering out negligible smaller forces that would otherwise introduce modeling errors.

Ensuring the inverse model has a unique solution

While it is straightforward to calculate the map from an applied force to proximal wrench, the inverse problem is often ill-posed, especially for two-point forces. To compare how well-posed the inverse mapping is for various robot designs we propose two metrics: local sensitivity and pairwise separation.

Local sensitivity

what do you mean by nearby?

Local sensitivity measures how much the outputs change for small input perturbations, quantified through Lipschitz continuity. This ensures local injectivity—that nearby wrenches map to distinct point forces, preventing the inverse model from confusing similar inputs. We can measure local injectivity using Lipschitz continuity.

A map $f : \mathbb{R}^n \mapsto \mathbb{R}^m$ is Lipschitz continuous on a domain if there exist constants $L > 0$ such that for all x_1, x_2:

$$L\|x_1 - x_2\| \leq \|f(x_1) - f(x_2)\|$$

If $L > 0$, then f and its inverse are locally injective and stable. The Lipschitz constants L can be approximated by the minimum singular values of the Jacobian over the domain. For the two point force case we would use the 6x6 Jacobian J of the forward map:

$$J(x;r) = \frac{\partial W}{\partial x} = \frac{\partial}{\partial[F_1, s_1, el_1, F_2, s_2, el_2]} \begin{bmatrix} T_x \\ T_y \\ T_z \\ T_x \\ T_y \\ T_z \end{bmatrix}$$

Using the minimum singular value of this Jacobian we have the following relationship:

what if 0?

$$\sigma_{min}(J)\|\Delta x\| \leq \|J\Delta x\|$$

Sampling multiple Jacobians across the entire force domain Ω provides an approximation for our Lipshitz constant L:

$$L \approx \min_{x \in \Omega} \sigma_{min}\big(J(x)\big)$$

If $L > 0$, the forward map is locally injective and stably invertible at the sampled points.

However, this metric does not account for global non-uniqueness (e.g., two far-apart configurations with same wrench)

Pairwise separation (updated 10/06/2025)

Pairwise separation checks that any two different inputs produce clearly different outputs. Imagine plotting every possible input force and its corresponding wrench output. If two very different force inputs give almost the same wrench point, the mapping is not unique — the inverse model can get confused. Pairwise separation measures that gap for every pair of inputs:

clarify

$$R_{ij} = \frac{\|W_i - W_j\|}{\|x_i - y_j\|}$$

what are x_i & y_j here?

- If R_{ij} is large, small input changes cause noticeable output changes → good injectivity.
- If R_{ij} is small, two far-apart inputs look almost the same → bad injectivity (ambiguous).

If we minimum R_{ij} across all pairs — this tells us the worst case "confusion" for a given design. We can then optimize the beam geometry so that this minimum ratio is as big as possible.

Questions: • Previously you had looked to max. stiffness gradient I think → what happened to this idea?
• How do you think an optimized geometry would compare to a naive design where radius just gets smaller at tip?

Optimized Geometry

We now explore abstract geometries and numerical methods for deriving optimal geometries for force sensing. Force estimation accuracy degrades significantly for two-point forces compared to single-point forces. Small deviations in measured wrench—from sensor noise, disturbances, or hysteresis—cause large changes in estimated forces. The optimizer often converges to suboptimal local minima due to an ill-conditioned, nearly singular system. To address this, we propose an optimization framework that improves injectivity through geometric design. We use two metrics: local sensitivity and pairwise separation. Both metrics appear in our objective function to produce designs that are locally stable and globally unique.

not sure I follow what you mean by this

Combined objective: The discretized formulation using N sampled inputs is

$$\max_{r \in \mathbb{R}^n} (\alpha \cdot \sigma_{\min} + (1-\alpha) \cdot \Delta_{\min})$$

where:

- local stability: $\sigma_{\min} = \min_{i=1,\dots,N} \sigma_{min}(J(x_i; r))$
- global injectivity (pairwise separation: $\Delta_{\min} = \min_{\substack{i,j=1,\dots,N \\ i \neq j}} \frac{\|W(x_i;r) - W_j(x_j;r)\|}{\|x_i - x_j\|}$
- weighting coefficient: α

subject to:

$$r_{min} \leq r_i \leq r_{max}, \quad i = 1, \dots, M$$

This method uses N sampled inputs $[\widetilde{F_1}, \widetilde{s_1}, \widetilde{el_1}, \widetilde{F_2}, \widetilde{s_2}, \widetilde{el_2}]$ and M consecutively connected link segments of uniform length with variable radii r_i.

The first term is our local stability metric. It computes the Jacobian of the forward map at each sample point using finite differences, then finds the minimum singular value across all Jacobians. The magnitude $\sigma_{\min}$ represents the smallest local gain of the forward map over all directions. Maximizing $\sigma_{\min}$ raises the lower bound on input distinguishability, ensuring the mapping remains injective and preventing inverse estimates from exhibiting extreme sensitivity to wrench errors.

The second term is our global stability metric. It compares the norm of input differences to wrench differences for all distinct sample pairs. The minimum ratio $\Delta_{\min}$ identifies the worst-case pair most likely to be confused. Maximizing $\Delta_{\min}$ raises the lower bound on input separability.